

Laboratoire de Programmation Temps Réel Semestre printemps 2025 - 2026

Laboratoire 1: Processus et threads Linux

Temps à disposition: 4 périodes

Laboratoire individuel

Objectifs pédagogiques

Durant ce laboratoire vous allez faire quelques mesures sur notre système.

Vous allez ensuite découvrir comment utiliser des processus et des threads sous Linux, comment contrôler leur exécution et quelques commandes spécifiques pour agir sur le comportement de l'ordonnanceur Linux.

Infrastructure des laboratoires

Tous les laboratoires se feront sur votre De1-SoC attribuée en début de semestre. Le développement se fera sur vos ordinateurs personnels (dans la VM) et/ou les machines de laboratoire en A07/A09.

Avant toute chose, veuillez préparer votre infrastructure en prêtant attention aux points suivants :

1. Installez le SDK sur votre machine à l'aide de l'archive fournie `de1soc-sdk.tar.gz`. Utilisez les commandes suivantes. Vous êtes libres d'installer le SDK où bon vous semble (tant que vous vous en souvenez). Il sera réutilisé tout au long du semestre

```
$ tar -xf de1soc-sdk.tar.gz
$ cd de1soc-sdk
$ ./relocate-sdk.sh
```

2. Pour compiler les applications, un fichier `CMakeLists.txt` vous est fourni. La génération des différents fichiers `Makefile` se fait en exécutant la commande suivante dans le dossier du laboratoire (remplacez `SDK_FOLDER` par le dossier où vous avez installé le SDK à l'étape précédente)

```
$ cmake -DCMAKE_TOOLCHAIN_FILE=<SDK_FOLDER>/share/buildroot/
  toolchainfile.cmake -S . -B build
```

3. Le déploiement des binaires à exécuter sur la DE1-SoC se fera au travers de `ssh`. Vous relierez donc votre machine de travail à la DE1-SoC par un câble Ethernet. L'interface ethernet,

de la DE1, est déjà configurée avec l'adresse 192.168.0.2. Ainsi, configurez l'interface ethernet de votre machine de travail de sorte qu'elle aie l'adresse 192.168.0.1 et le masque 255.255.255.0. Si tout se passe bien, vous devriez pouvoir *ping*er ou vous connecter en *ssh* à la DE1 à l'adresse 192.168.0.2 (nous utiliserons le compte root, lequel n'a pas de mot de passe).

```
$ ping 192.168.0.2
$ ssh root@192.168.0.2
```

4. Testez votre configuration en lançant la compilation :

```
$ cmake --build build
```

5. Vous devriez voir les programmes compilés dans le dossier `build` généré dans le dossier du labo en cours. Vous pouvez utiliser les commandes `scp` et `ssh` à la main pour respectivement déployer et exécuter les exécutables sur la carte (utilisez les pages *man* pour comprendre comment les utiliser) mais nous vous fournissons également le script `delsoc.sh` qui vous permet d'effectuer les actions suivantes

```
$ ./delsoc.sh deploy <prog_name> # Deploie le programme sur la carte
$ ./delsoc.sh run <prog_name> # Deploie puis lance le programme sur la carte
$ ./delsoc.sh kill <prog_name> # Termine le programme sur la carte
```

Vous pouvez aussi ouvrir une session sur le port série avec l'utilitaire `picocom`. Connectez le câble pour le port série (le câble USB A vers mini-USB fournie avec votre DE1) sur votre machine de travail et le port mini-USB de la DE1, puis tapez :

```
$ sudo picocom -b115200 /dev/ttyUSB0
```

1ères mesures

- ✖ 1. Complétez le programme (`cpu_loop.c`) qui compte le nombre d'itérations d'une boucle exécutée durant `N_SEC` secondes;
Note: `N_SEC` doit être définie par la directive `#define`
- ✓ 2. Le logiciel doit également imprimer en sortie le nombre d'itérations effectuées par seconde
- 3. Compilez avec `cmake -build build`
- 4. Déployez et exécutez sur la DE1 avec `./delsoc.sh run build/cpu_loop`

Le cœur du programme devrait ressembler à quelque chose comme

```
start_time = time(NULL);
nb_iterations = 0;
while (time(NULL) < (start_time + N_SEC)) {
    ++nb_iterations;
}
```

Exemple de boucle

Vous aurez probablement besoin des headers suivants, qui sont déjà déclarés dans le fichier fourni:

Headers utiles

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/time.h>
```

Si vous avez besoin d'information sur la fonction `time()`, exécutez `man 2 time`

- ✂ 5. Modifiez votre programme à l'aide de macros afin de compter respectivement le nombre d'opérations `*` et `/` effectué par seconde sur une donnée de type `float`;
- ✓ 6. Commentez les résultat des nombres d'opérations observés par seconde. Que pouvez-vous en déduire ?

Multi-tâche: Commandes utiles

Linux est un système d'exploitation moderne et multi-utilisateurs. Il est multi-tâche et impose la séparation entre deux mondes: l'espace utilisateur (*user space*) et l'espace noyau (*kernel space*). Lors de la première manipulation vous avez travaillé dans l'espace utilisateur, en écrivant et exécutant de simples programmes. Cette manipulation va vous permettre de plonger dans l'interface des deux mondes. Premièrement, le noyau offre de nombreuses informations à propos de l'état du système dans des fichiers virtuels situé dans le dossier `/proc`. Par exemple, `/proc/cpuinfo` permet d'afficher des informations que le noyau connaît à propos du ou des processeurs physiques. La commande `cat` permet d'afficher le contenu d'un fichier sur la sortie standard. Essayez donc la commande suivante:

```
$ cat /proc/cpuinfo
```

Cette commande devrait vous fournir des informations utiles à propos du processeur de la machine.

Si la machine est un ordinateur multi processeur ou multi-core, la commande `man taskset` vous apprendra comment forcer l'exécution d'un processus sur un coeur spécifique.

Une autre commande utile pour vérifier le statut d'un processus et plus généralement de la machine est `ps`. N'hésitez pas à utiliser la commande `man` pour en apprendre plus à son sujet. Exécutez:

```
$ ps aux
```

sur votre machine de travail. Cette commande vous permet d'observer le statut des processus en cours d'exécution: Que signifient les lettres et symboles de la colonne `STAT`? Etablissez une petite table contenant les symboles, l'état du processus et une brève description de leur sens.

Une autre commande vous permet d'observer dynamiquement l'évolution du système: `top`.

Ordonnancement en temps partagé

Linux est un système à temps partagé (*shared time system*). Commençons par le démontrer¹. Ces expériences seront plus pertinentes en n'exploitant qu'un seul CPU. La commande `taskset` vous permettra de forcer l'utilisation d'un seul coeur.

¹Pour ces expériences nous allons réutiliser le code du premier point (`cpu_loop.c`)

```
$ taskset -pc 0 $$  
$ ./cpu_loop
```

Essayez maintenant d'exécuter

```
$ ./cpu_loop & ./cpu_loop
```

```
$ ./cpu_loop & ./cpu_loop & ./cpu_loop
```

```
$ ./cpu_loop & ./cpu_loop & ./cpu_loop & ./cpu_loop & ./cpu_loop
```

- ✓ Pouvez-vous expliquer le résultat de ces trois commandes? Si vous avez des doutes, ajoutez `getpid()` la sortie des `printf` de `cpu_loop.c`.
Si vous ouvrez 2 shells et utilisez la même commande `taskset` dans chacun, et exécutez ensuite la commande suivante sur le premier shell:

```
$ sleep 1; ./cpu_loop & ./cpu_loop
```

Et au même instant la commande suivante sur le second:

```
$ ./cpu_loop & ./cpu_loop
```

- ✓ Quel résultat obtenez-vous? Pourquoi?

Migration de tâches

Une fois en exécution, un processus est déplacé par le système d'exploitation sur les différents CPUs (tout en respectant l'affinité imposée par la commande `taskset`) pour mieux répartir la charge. La fonction `sched_getcpu()` est une extension spécifique à Linux qui permet de voir sur quel CPU la tâche qui a appelé la fonction est en exécution.

- ✱ Écrivez un programme (`get_cpu_number.c`) qui affiche à l'écran le numéro du CPU sur lequel il tourne au démarrage et chaque déplacement qu'il subit (avec une indication de l'heure à laquelle le déplacement se produit).
- ✓ Exécutez ensuite, à l'aide de la commande `taskset` précédemment vue, le logiciel `cpu_loop` sur le même CPU où `get_cpu_number` tourne. Qu'est-ce qu'il se passe? Utilisez la commande `htop` pour visualiser la charge du système et commentez par écrit.

Le système a été configuré afin d'isoler le CPU 1. Cela a été fait en ajoutant `isolcpus=1` au `bootargs` de Linux. On peut le voir dans les messages initiaux lors du boot de Linux:

```
[ 0.000000] Kernel command line: console=ttyS0,115200 root=/dev/mmcblk0p3 rw rootwait
earlyprintk isolcpus=1 nohz_full=1
```

Déactivez cette isolation en suivant les instructions suivantes et relancez les tests effectués précédemment.

1. Entrer dans le bootloader u-boot en stoppant l'autoboot de linux en pressant une touche lorsque le message suivant apparaît Hit any key to stop autoboot;;
2. Depuis u-boot, configurer les bootargs et démarré Linux:

```
=> setenv bootargs "console=console=ttyS0,115200 root=/dev/mmcblk0p3 rw rootwait
earlyprintk"
=> boot
```

Les priorités et niceness

Grâce à la commande `nice` il est possible de définir la niceness d'un processus. Lisez le manuel et lancez l'expérience précédente en exploitant `nice`. Voici un exemple:

```
$ nice -n 5 ./cpu_loop & ./cpu_loop
```

- ✓ Commentez les résultats.

Codage

- ✓✂ La gestion des priorités et l'interfacage avec l'ordonnanceur sont également mises à disposition du programmeur. En utilisant la bibliothèque *PTHREAD*, complétez les fonctions du code suivant. Exécutez-le et commentez le code et les résultats. Le programme crée `n_threads` threads avec différentes priorités et, à la fin de l'exécution, affiche le nombre de cycles exécutés par chaque thread. Le coeur de la fonction `f_thread` est le même que celle de `cpu_loop.c`.

N'hésitez pas à consulter l'aide de

```
int pthread_setschedprio(pthread_t thread, int prio);
pthread_t pthread_self(void);
int pthread_barrier_wait(pthread_barrier_t *barrier);
```

Fonctions utiles

- ✓ Quelles sont les différences entre niceness et priorités?

Travail à effectuer

1. Le laboratoire se fait de manière individuelle.
2. Codez les programmes demandés (symbole ✂).

3. Notez les réponses aux questions indiquées par le symbole ✓. Nous en débattons lors d'une séance ultérieure.
4. Rendez le tout sur cyberlearn, dans un fichier compressé nommé `rendu.tar.gz`
 - Ce fichier doit être généré en lançant le script `ptr_rendu.sh`
 - Le script vérifie qu'un fichier `rapport.pdf` est présent à son niveau, ainsi qu'un dossier `code` également présent au même niveau

Les connaissances acquises durant les laboratoires seront également testées lors des tests écrits ainsi que lors de l'examen final.